Available online at www.sciencedirect.com

ScienceDirect

IFAC PapersOnline CONFERENCE PAPER ARCHIVE

IFAC PapersOnLine 55-4 (2022) 248–253

Implementation of Robotic Kinematics Algorithm for Industrial Robot Model Using Microcontrollers

Karol Borowski * **Andrzej Wojtulewicz** **

* *Institute of Control and Computation Engineering, Warsaw University of Technology, ul. Nowowiejska 15/19, 00-665 Warsaw, Poland, Email: kborowsk@mion.elka.pw.edu.pl*

** *Institute of Control and Computation Engineering, Warsaw University of Technology, ul. Nowowiejska 15/19, 00-665 Warsaw, Poland, Email: andrwoj@gmail.com*

Abstract: This paper aims to describe the implementation of hardware and software layers for a physical model of an industrial robot. The discussed manipulator has three degrees of freedom, which are responsible for positioning the end effector in the XYZ coordinate system and one degree of freedom controlling a rotation of the end effector. Both manual control and automatic robot movement to the desired path are considered. For this purpose, the STM32F4 microcontroller is used. This work describes the following issues: the choice of appropriate components, the design of secure PCB boards, implementation of the software layer. It consists of the microprocessor logic layer responsible for handling peripherals and safely controlling the mechanical arm. Handling the robot manipulator is possible thanks to the created graphical user interface, which communicates with the logic layer.

Copyright © 2022 The Authors. This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0/)

Keywords: Robotic manipulator, Forward kinematics, Inverse kinematics, PID controller, Mechanical arm, Jacobian matrix, Microcontroller

1. INTRODUCTION

Robotic manipulators play a significant role in modern industry. They are mechanical arms that, like human arms, are composed of members Resti et al. (2018). In the case of a human arm, such a member is a shoulder, forearm, hand, or a single finger. The members are connected by joints that allow them to change position. In mechanics, these joints are called hinges. It is impossible to imagine more prominent production lines today without manipulators. They perform tasks that are too dangerous for humans (working in a chemical environment) and too difficult or even impossible to perform by humans (handling weighty and bulky objects). They are also perfect for monotonous tasks, for which it is not worth employing a man, e.g., moving products from one production line to another or sticking labels on objects. Due to the very high capabilities of these machines, it is possible that soon, they will be developed in other industries closer to everyday life. Mechanical arms have also been used in surgery since the beginning of the 21st century Bouteraa et al. (2018).

The primary objective of this work is to develop a control system for a robot with four degrees of freedom, allowing it to move in the XYZ coordinate system Satya Durga Manohar Sahu et al. (2021). In addition to the system layer, it is planned to design the hardware layer, i.e., a printed PCB. This is to avoid the need for prototyping boards and cables, which in larger projects become inconvenient to use and unreliable. The logic layer is decided to be implemented on a microprocessor system. It enables control of the robotic manipulator in two modes - manual and automatic. The user should be able to read and modify the system parameters.

The intermediate goal of the work is to create a ready-to-use runtime kit together with a library containing functions allowing rapid prototyping of solutions (both hobby and commercial) based on a similar robotic manipulator Zhao and Hong (2021). The main advantage of this solution is the ability to quickly run the essential functions of the robot without having to implement the control algorithm from scratch. However, creating a runtime kit that fully meets its objectives is beyond the scope of this paper, so this idea is left for future development and improvement. Two practical methods for implementing kinematics are presented in the paper Jokić et al. (2020) using PC as well as programmable logic circuits.

2. CONTROL ALGORITHM

The main objective of this work is to implement the kinematics of an industrial manipulator with four degrees of freedom. Robot kinematics are mathematical equations that determine the relationship between each joint of the robot and the position of the actuator, or effector, located at the end of the last member. The authors of the paper Hayat et al. (2015) described and compared in their work the two most popular methods of moving the effector along a given trajectory. The test object was a robot with six degrees of freedom. The first method is the analytical method Pieper (1968). This method is based on solving the

2405-8963 Copyright © 2022 The Authors. This is an open access article under the CC BY-NC-ND license.
Peer review under responsibility of International Federation of Automatic Control.
10.1016/j.ifacol.2022.06.041

inverse kinematics equations of the robot, i.e., determining such positions of individual members for which the effector is at a given point.

The second method described uses the Jacobi matrix Jacobi Goldenberg et al. (1985)Manseur and Doty (1989), or more precisely its inverse. In robotics, the Jacobi matrix is a derivative matrix that defines the relationship between the change in position of the effector and the change in position of the individual joints of the robot over short time intervals Δt. In other words, this matrix defines the relationships between the joints' velocities and the effector's resultant velocity.

With the Jacobi matrix, the resultant velocity of the effector can be determined by knowing the velocities of the individual members. However, this is not the purpose of inverse kinematics. The inverse Jacobi matrix must be determined to calculate the change in the joint position needed to set the effector in motion at a given velocity. As with the previous method, to guide the effector along a specific path, the path must be divided into smaller fragments. The difference is that in the analytical method, the exact joint positioning that positions the effector at a given point was calculated. In the case of the inverse Jacobi matrix, one calculates the $\Delta\theta$, which is the change one must make to the current position to bring the effector closer to the target point. The solution will be accurate enough if the appropriate refresh rate of the target position is chosen.

Each of the compared methods has its advantages and disadvantages. The analytical method has very high accuracy at the expense of computation speed. The algorithm in this paper was implemented on a microprocessor with limited computing power, so the execution time of the kinematics calculations was an important issue. The processor had to be fast enough to execute the inverse kinematics algorithm, drive the four motors, handle communication with the user, and even send data to an external computer in a reasonable time frame. In this respect, the inverse Jacobi matrix method is a better algorithm.

Additionally, the analytical method relies on controlling the set positions of individual joints. Since this work uses DC motors, controlling minimal position changes over short time intervals would not be accurate (stepper motors would be much more suitable for this). On the other hand, the inverse Jacobi method allows the position of the effector to be controlled by the instantaneous set velocities of the individual joints, which is both more accurate and easier to do with DC motors.

The robot that is the subject of this work has 3 degrees of freedom, controlling the effector's position. Each joint has a minimum angle q_{min} and a maximum angle q_{max} that it can reach. Based on these, it is possible to calculate the space in the Cartesian coordinate system over which the effector can move. However, the constraints on the motion of the robot's joints are not all that define the possible position of the robot. The robot's environment itself, e.g., a wall, the ground, or other types of obstacles, must also be considered. The authors of the paper Oetomo et al. (2009) describe how to determine the working area of a manipulator under appropriate constraints. Their method is to divide the work area into smaller areas and evaluate whether or not a given fragment is reachable by the effector. In this paper, the workspace is defined only by joint constraints. No additional constraints are imposed. The robot is assumed to operate in a space with no physical obstacles. The only constraint could be a collision with the ground, but the range of motion of the joints does not allow for this possibility. Additionally, the system was equipped with limit sensors to protect the joints from leaving their allowable limits. Thus, the system would be able to operate safely even without defining a working area.

The book Spong et al. (2004) is a valuable resource in writing the work. The authors have very reliably described the most critical aspects of robotic manipulators. This subsection is mainly based on the knowledge acquired from the above authors. Two coordinate systems are used to determine the position of the manipulator. The first is an internal coordinate system consisting of a vector of variables $q = (q_1, q_2, q_3, ..., q_n)^T$. In the case of a rotary joint, this is the angle by which the joint has rotated relative to the base position, so $q_i = \theta_i$. As for prismatic joints, $q_i = d_i$ is the displacement distance relative to the base point.

The position of each joint relative to the others in a unit distance is defined by the external coordinate system. It defines the effector's position in Euclidean space, which is expressed by the X, Y, Z axes. Each degree of freedom has its coordinate system, relative to which the position of subsequent degrees is determined. The effector's position is determined relative to the first coordinate system, the base system.

In order to perform the transformation from the inner coordinate system to the outer coordinate system and vice versa, it is necessary to determine the relationships between the joints, i.e., how a change in q_i affects the position of the outer coordinates of subsequent joints. The homogeneous transformation matrix and Denavit-Hartenberg notation are used for this purpose. A single homogeneous matrix has the form

$$A_i = \begin{bmatrix} R_i^{i-1} & O_i^{i-1} \\ 0 & 1 \end{bmatrix}, \tag{1}$$

where R is the rotation matrix and O is the translation matrix from $i-1$ to i.

The matrix in equation (1) is not constant. The rotation and translation matrices are functions whose argument is the aforementioned variable q_i. The single matrix A_i is thus a function depending on q_i

$$A_i = A_i(q_i). \tag{2}$$

Having determined the transformation matrices between adjacent members, one can determine the rotation and position of the effector concerning the underlying coordinate system. For this purpose, the relations between particular homogenous matrices are used

$$T_j^i = A_{i+1}A_{i+2}\dots A_j = \begin{bmatrix} R_j^i & O_j^i \\ 0 & 1 \end{bmatrix}, \text{if } j > i. \tag{3}$$

In this case, the transformation matrix T_j^i expresses the position and orientation of the system j concerning i. The positions of the executive are defined by the matrix

$$H = T_n^0 = A_1(q_1)A_2(q_2)\dots A_n(q_n) = \begin{bmatrix} R_n^0 & O_n^0 \\ 0 & 1 \end{bmatrix}, \quad (4)$$

where n is the number of degrees of freedom.

The D-H notation is based on maintaining appropriate criteria when determining the coordinate systems of the individual joints. The following requirements must be met:

- The x_i axis is perpendicular to the z_{i-1} axis,
- The x_i axis axis intersects with the z_{i-1} axis.

In addition, the axis z_i is defined by the joint $i+1$. For rotary joints, z_i is the axis of rotation of joint $i+1$, and for prismatic joints, the axis z_i coincides with the displacement of joint $i+i$. The axis y_i, as well as the direction of the rotation angle, are determined using the right-hand rule.

Using D-H notation, the homogeneous transformation matrix is expressed by the formula (5)

$$A_i = R_{z,\theta_i} Trans_{z,d_i} Trans_{x,\alpha_i} R_{x,\alpha_i} = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i}c_{\alpha_i} & s_{\theta_i}s_{\alpha_i} & a_i c_{\theta_i} \\ s_{\theta_i} & c_{\theta_i}c_{\alpha_i} & -c_{\theta_i}s_{\alpha_i} & a_i s_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (5)$$

By satisfying the D-H criteria, the four parameters in the above formula are associated with the i-th member and the i-th joint:

- a_i - the distance between z_{i-1} and z_i measured along the x_i axis,
- α_i - The angle between z_{i-1} and z_i with respect to the x_i axis,
- d_i - distance between the center of the O_{i-1} coordinate system and the intersection of the x_i axis with z_{i-1}measured along the z_{i-1} axis,
- θ_i - the angle between x_{i-1} and x_i with respect to the z_{i-1} axis.

Following the rules described above, a drawing of the manipulator, which is the subject of the work, was created.

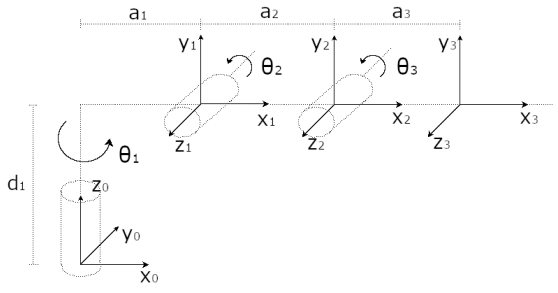


Fig. 1. D-H notation compliant manipulator

Based on Figure 1, the members of the actual robot model were measured and a table of D-H parameters was made (Table 1).

Table 1. D-H notation parameter table

lp.	θ [rad]	α [rad]	a [cm]	d [cm]
1	θ_1	$\frac{\pi}{2}$	7.0	22.0
2	θ_2	0.0	24.0	0.0
3	θ_3	0.0	18.5	0.0

In order to calculate the transformation matrix, the MATLAB environment was used, and then the obtained results. The results were saved in the uC program. Calculations were performed on parameters in the form of symbols. The exact values contained in Table 1 were substituted only in the microcontroller program code. Thanks to such a solution, correction of wrongly measured parameter came down only to change the given constant value. Otherwise, we would have to recalculate the transformation matrix in MATLAB.

To determine the position of the effector concerning the base system, we only need the vector O_n^0. The rotation, in this case, is not essential because the manipulator has such limited joint motions that there is always only one solution to the inverse kinematics task. A Jacobian algorithm is chosen to solve the inverse kinematics task, defining the relationships between the velocity of the effector and the velocities of the individual joints $\dot{q}(t)$. In the case of the present manipulator, the Jacobi matrix is a 3 x 3 square matrix, and due to mechanical joint constraints, the robot has no singular points i.e., points for which $\det J(q) = 0$. The Jacobian algorithm is presented in the form of equations (6)(7)

$$\begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ \dot{\theta}_3 \end{bmatrix} = J^{-1} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix}, \quad (6)$$

$$\begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \\ \dot{\theta}_3 \end{bmatrix} = \begin{bmatrix} \mathrm{inv}J_{11} & \mathrm{inv}J_{12} & \mathrm{inv}J_{13} \\ \mathrm{inv}J_{21} & \mathrm{inv}J_{22} & \mathrm{inv}J_{23} \\ \mathrm{inv}J_{31} & \mathrm{inv}J_{32} & \mathrm{inv}J_{33} \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix}. \quad (7)$$

As in the simple kinematics task, calculations were performed in the MATLAB environment on constants in the form of symbols. The equation 7 was determined and implemented in the uC program as a system of equations.

One of the goals of this study was to move the effector along a straight line automatically, and only such a trajectory was implemented in the uC program. For the motion to follow the specified trajectory, the component velocities of the effector along each axis had to be calculated. The formulas used for this were (8)

$$\begin{aligned} \dot{x} &= \frac{v(x_2 - x_1)}{\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2 + (z_2-z_1)^2}}, \\ \dot{y} &= \frac{v(y_2 - y_1)}{\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2 + (z_2-z_1)^2}}, \\ \dot{z} &= \frac{v(z_2 - z_1)}{\sqrt{(x_2-x_1)^2 + (y_2-y_1)^2 + (z_2-z_1)^2}}, \end{aligned} \quad (8)$$

where $[x_2, y_2, z_2]^T$ is the target point, $[x_1, y_1, z_1]^T$ the current point, and v the resultant velocity of the effector.

The new velocity vector $\dot{q}$ is updated at a frequency of 100Hz. To keep the effector moving in a straight line, not only the inverse Jacobi matrix but also the value of the vector $[\dot{x}, \dot{y}, \dot{z}]^T$ is recalculated at each iteration.

3. DESCRIPTION OF HARDWARE

The robot model has to be retrofitted (Figure 2). It has old motors and gears which were replaced with new ones. The robot model has 3 degrees of freedom, defining the robot's

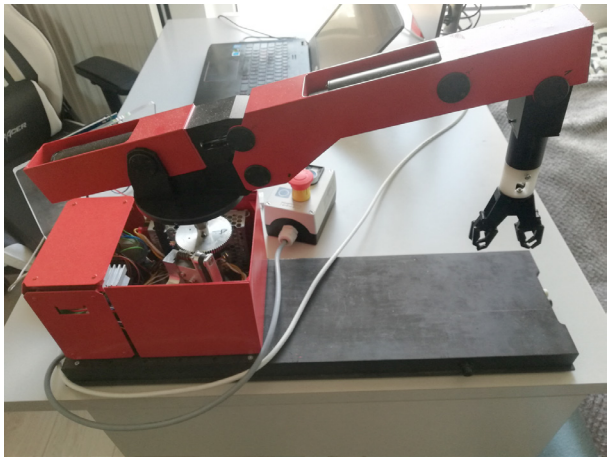


Fig. 2. Robot model

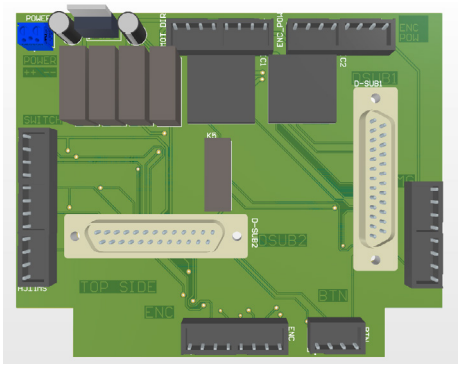


Fig. 3. Front side of the interface board

position in XYZ space. The first one is responsible for the rotation around the vertical axis, and the next two control the rotation around the horizontal axis. In addition, the robot has a fourth degree of freedom which is responsible for effector rotation. It is also possible to connect a motor that controls the clamping of the gripper, but this option was not considered in this work. One of the objectives of the present work is to design the hardware layer. It consists of two boards designed in an Altium environment. The interface board is responsible for collecting all the signals from the external components and transferring them to the second proprietary board, which is an overlay to the STM32 Nucleo.

The interface board is designed to collect all the signals in one place to keep the wiring controlled and organized. It connects directly to the overlay for the Nucleo so that the processor has access to all peripherals. Cables from individual components are connected to terminal strips inserted into sockets on the board. This makes it possible to disconnect and connect specific plugs seamlessly. All THT through-hole-mount components are placed on the front side of the board (Figure 3), and SMD surface-mount components are placed on the backside. The board's dimensions are chosen to make it as large as possible to fit into the robot case, hence its irregular shape.

After reviewing the example solutions and creating an initial concept for manipulator control, a list of assumptions and functionalities that the system should meet is compiled:

Main assumptions and functionalities of the system:

- Operating the robot in manual mode in two ways - controlling a single joint or moving the effector along the XYZ axis.
- Operation in automatic mode - when moving the effector in manual mode, the user has the ability to add the current gripper coordinates to the processor memory and put the robot into automatic motion between stored points.
- In automatic mode, the effector moves between points in a straight line. When this is not possible, an appropriate message is shown to the user.
- It is possible to delete a saved trajectory.
- The user can delete a saved trajectory.
- The current status of the robot is indicated.
- Protection against manipulator damage.
- Edit system parameters such as effector speed, joint speed, speed along XYZ axis.
- Restricting using an external computer only to collect data from the system.
- The runtime board with the processor connects to the system components indirectly through a proprietary PCB interface board.

Knowing the detailed requirements of the system, there is a selection of appropriate components to meet those requirements. For the main controller the STM32 Nucleo-64 kit with the STM32F401RE Cortex-M4 84MHz processor is selected. One of the paper's objectives is the user operation of the device. In order to keep the concept of limiting access to the computer and to be able to display the status of the system, it is decided to use a touch screen LCD for this purpose. It communicates with the CPU, just like a PC, via a UART interface. As a touch screen display, the STM32F769I Discovery runtime kit with the STM32F769NIH6 microcontroller is used. An essential advantage of the kit is the support of the TouchGFX program. It is an environment that facilitates the creation of high-level graphical interfaces. Its key feature is the ability to simulate the program on a computer without loading it into the processor memory and quite a convenient implementation of high-quality solutions.

4. IMPLEMENTATION OF AN EMBEDDED SYSTEM FOR ROBOT CONTROL

The microcontroller program was written in C language. The STM32CubeIDE environment was used for this purpose, allowing quick and simple configuration of the processor peripherals and code generation with preset settings. Solving the problem of this work required adherence to a strict time frame, so the real-time operating system freeRTOS was used.

The graphical interface was implemented on the STM32F7 Discovery development kit using the TouchGFX environment developed by STMicroelectronics. It enabled convenient addition of graphics, components and generation of code, which was then uploaded to the STM32F769I memory. Part of the code responsible for the low-level operation of the uC was generated in C, while the display operation was generated in C++. During the work, the dedicated documentation of the environment was used, which describes all available components and examples of how to

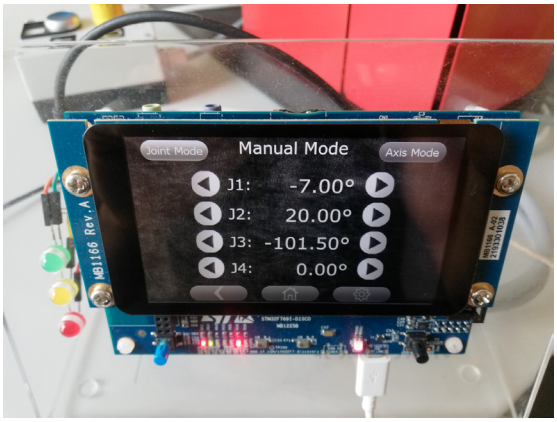


Fig. 4. Manual mode - joint control

use them. However, by itself, TouchGFX only allows to place components on the screen and implement simple interactions on events. For more complex operations, a given event (such as pressing a button) generates a virtual function that must be manually overwritten in C++. For this purpose, the same environment used to implement the Nucleo board software was used. The sample screens included in the GUI are described below. Manual mode, joint control (Fig. 4) in which the user can control the individual joints of the manipulator separately using the arrow keys. The current values of the internal variables, given in degrees, are displayed between the buttons. To change the mode to control the effector along the axis of the coordinate system, press the "Axis Mode" button.

5. RESULTS OF EXPERIMENTS

According to work, the robot can be controlled in three ways. After the implementation of the algorithms, their quality was tested. The tests were performed using the computational environment MATLAB. While controlling the manipulator, the system sent the necessary data to MATLAB through the UART interface, which were then presented as graphs depending on the number of samples t/T_s, where T_s is the sampling period equal to 10ms.

The manual control mode involves moving the effector along one of the third axes of the coordinate system. Tests are performed on each axis for different actuator speeds. The criterion for control evaluation is graphs showing the differences between the actual path of movement and the expected one. The error is calculated from the difference in the distance of each point of the robot's path from the straight line, which is the expected path of motion.

Testing the motion along the X-axis, two different effector positions concerning the Y-axis are checked. The first case is the motion as far as possible from the Y-axis, then the first joint plays a significant role during the motion. The motion path for velocity V = 1 cm/s has a somewhat significant motion deviation from the expected path. After traveling about 5cm, the error was as high as 1cm. For the same effector velocity, when moving close to the Y-axis, the deviation from the expected trajectory is much smaller. After traveling a similar distance, the error was less than 0.2 cm. The difference is due to the involvement of the first joint during steering. During operation, it is controlled with a relatively small PWM signal instead of

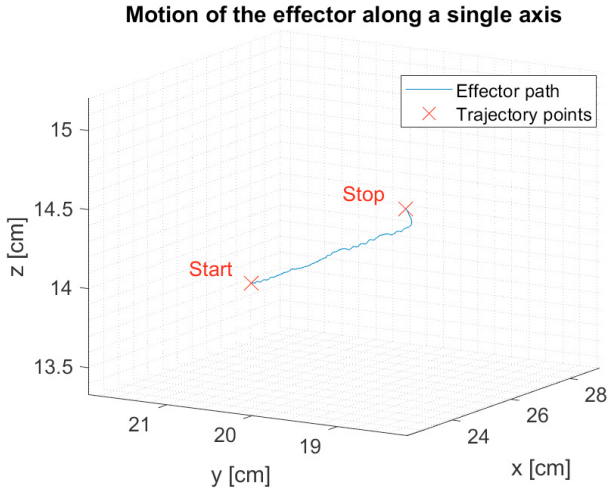


Fig. 5. Effector motion path speed 5.0 cm/s

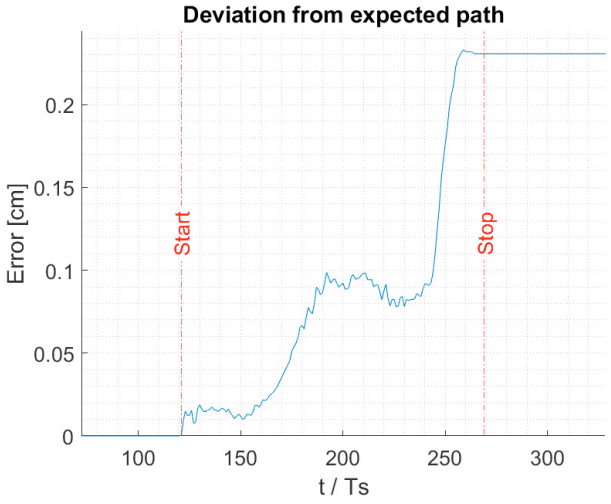


Fig. 6. Control error speed 5.0 cm/s

the second and third joints, so the lower the velocity it has to achieve, the lower the accuracy as it moves at the stopping limit. For motion along the X-axis assuming Y = 0, the first joint remains stationary, so in this case, the motion of the effector is much more accurate. By increasing the velocity of the effector while moving away from the Y-axis to 5 cm/s (Figure 5), a much more accurate result was achieved than with five times less velocity. This time, after traveling about 5 cm, the error is 0.23 cm (Figure 6) because the first joint of the robot is controlled at a higher speed, resulting in a more accurate adjustment.

The assumption of the automatic mode is to move the effector along a straight line. Different trajectories are tested for different effector resultant velocities. The control accuracy is evaluated using the same criterion as for manual control of the effector along the XYZ axis. Unlike the motion along the XYZ axis, the deviation of the motion from the expected path decreased close to the target motion point. The algorithm continuously selects the joint velocity values to reach the target position as closely as possible to the set trajectory. A trajectory consisting of three points is tested for effector velocities of 1 cm/s, 2.5 cm/s and 5 cm/s (Figure 7). The best result for a given trajectory was achieved at 2.5 cm/s.

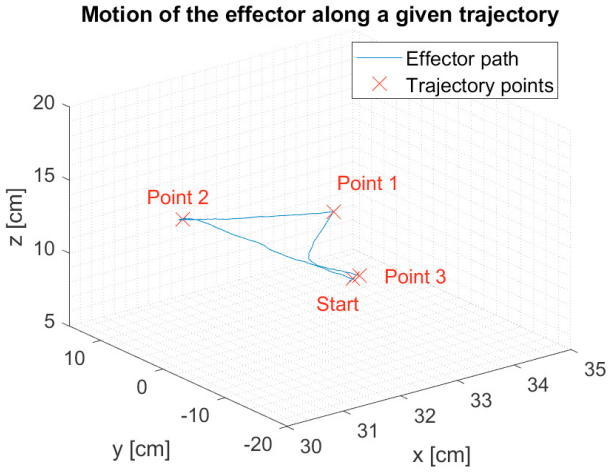


Fig. 7. Effector motion path speed 5 cm/s

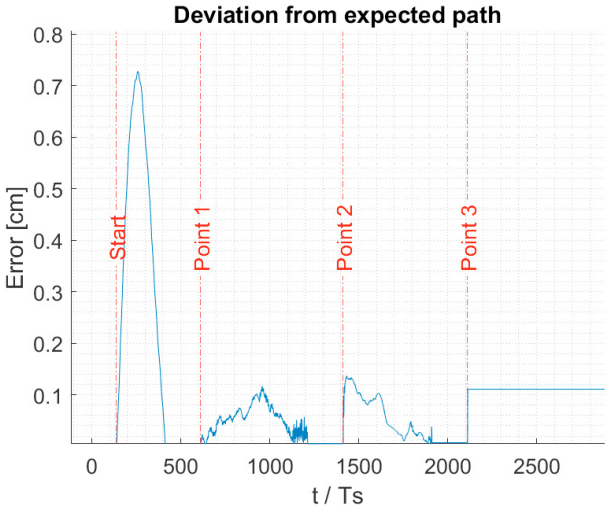


Fig. 8. Control error speed 5 cm/s

The larger control errors for speeds of 1 cm/s and 5 cm/s (Figure 8) are due to the same reasons described in manual mode. The trajectory segments between points 1-2 and 2-3 mainly consist of X-axis motion, so errors at these stages are larger for lower speeds and smaller for higher speeds. In contrast, the segment between the start and point one is mainly vertical, so the adjustment on this segment is more accurate for lower effector velocities.

6. CONCLUSION

In this paper, two control modes of an industrial manipulator are realized - manual mode and automatic mode. The manual mode is divided into two control modes - moving the effector along the axis of the coordinate system and moving the individual joints of the robot. The automatic mode allows the effector to move spontaneously in a straight line between points added to the trajectory. The robot is driven by four DC motors, which are controlled by PID controllers.

The main assumptions of the work regarding the control of the manipulator have been fulfilled. Different variants of manual control and an automatic mode based on two trajectories are tested. It is shown that the accuracy of the manipulator control is dependent on the set effector velocity. It cannot be too low because then the accuracy of joint velocity control decreases. On the other hand, if the speed is too high for some trajectories, there is a risk that the required speed of the slowest joint is not reached by it. Both situations described above result in a significant deviation of the effector trajectory from the expected path. Results of the control process are very satisfactory, considering the chosen method for calculating the inverse kinematics of the manipulator and the use of DC motors, which, compared to stepper motors, are more difficult to control with high precision.

An intermediate goal of the work is to create a runtime kit and a library to run the manipulator quickly. Functions are implemented to control the rotation of the effector, setting the effector at a given position and moving it along the XYZ axis. The PCB boards allow to connect to the system and program any Nucleo 64 board. The sockets allow connecting any DC motor not consuming more than 1A current and supplied with 12V voltage.

REFERENCES

Yassine Bouteraa, Ismail Ben Abdallah, and Jawhar Ghommam. Task-space region-reaching control for medical robot manipulator. *Computers and Electrical Engineering*, 67:629–645, 2018. ISSN 0045-7906.

A. Goldenberg, B. Benhabib, and R. Fenton. A complete generalized solution to the inverse kinematics of robots. *IEEE Journal on Robotics and Automation*, 1(1):14–20, 1985.

A. Hayat, R. Sadanand, and S. Saha. Robot manipulation through inverse kinematics. pages 1–6, 07 2015. doi: 10.1145/2783449.2783497.

Dejan Jokić, Slobodan Lubura, Vladimir Rajs, Milan Bodić, and Harun Šiljak. Two open solutions for industrial robot control: The case of puma 560. *Electronics*, 9(6), 2020.

Rachid Manseur and Keith Doty. A robot manipulator with 16 real inverse kinematic solution sets. *I. J. Robotic Res.*, 8:75–78, 10 1989. doi: 10.1177/027836498900800507.

Denny Oetomo, David Daney, and Jean-Pierre Merlet. Design strategy of serial manipulators with certified constraint satisfaction. *IEEE Transactions on Robotics*, 2009.

Donald Lee Pieper. *The Kinematics of Manipulators Under Computer Control*. DTIC Document, 1968.

Yulia Resti, Amrifan Mohruni, Firmansyah Burlian, Irsyadi Yani, and A Amran. Design of mechanical arm for an automatic sorting system of recyclable cans. *Journal of Physics: Conference Series*, 1007:012066, 04 2018.

Venkata Satya Durga Manohar Sahu, Padarbinda Samal, and Chinmoy Kumar Panigrahi. Modelling, and control techniques of robotic manipulators: A review. *Materials Today: Proceedings*, 2021. ISSN 2214-7853.

Mark W. Spong, Seth Hutchinson, and M. Vidyasagar. *Robot Dynamics and Control Second Edition*. 2004.

Qiangqiang Zhao and Jun Hong. An analytical framework for local and global system kinematic reliability sensitivity of robotic manipulators. *Applied Mathematical Modelling*, 102:331–350, 2021. ISSN 0307-904X.